Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs

DAY 1

Foundation

Setup, models, migrations, admin, ORM, and persistence

Sunday, October 25, 2026

Your five-day journey

One project grows all week: the Task Management API

01

Foundation

Project, Task model, migrations, admin

TODAY

02

Core API

CRUD endpoints and validation

03

Security

Tokens, ownership, isolation

04

Quality

8 tests, settings from the environment

05

Publish

Deploy, verify, README, live demo

Today's session

About three hours: learn it, practice it, build it

Explain + live demo
~60 min

Exercise
~60 min

Break
short

Project
~60 min

Exit check
end of day

Priority rule: finish the exercise core, then move to the project. Stretch work can wait.

Learning outcomes

By the end of today you can...

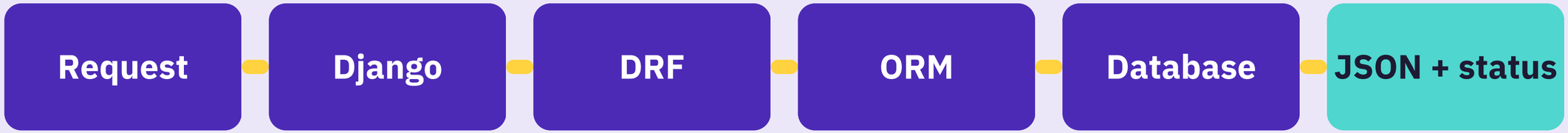
01 Explain the HTTP request–response flow

02 Run a reproducible Django REST Framework environment

03 Persist Task records through a correctly migrated data model

The finished project first

Start with the request, then trace what Django does with it




```
GET /api/tasks/7/   → 200 OK
{"id": 7, "title": "Write README", "status": "TODO"}
```

The URL names the resource. The verb names the action.

HTTP verbs on the Task resource

One resource, five actions

Verb	URL	Action	Success
`GET`	`/api/tasks/`	reads the list	200 OK
`POST`	`/api/tasks/`	creates a task	201 Created
`GET`	`/api/tasks/{id}/`	reads one task	200 OK
`PUT / PATCH`	`/api/tasks/{id}/`	changes a task	200 OK
`DELETE`	`/api/tasks/{id}/`	removes a task	204 No Content

Errors you will meet: **400 invalid input** **401 no token** **404 not found**

Project vs app

How a Django codebase is organized

```
task-management-api/
├── config/   # the project
|   ├── settings.py
|   └── urls.py
├── tasks/    # the app
|   ├── models.py
|   └── admin.py
└── manage.py
```

config is the project

Settings, root URLs, and the WSGI entry point

tasks is the app

Models and admin today; serializers and views from Day 2

Django provides models, the ORM, admin, and users. DRF sits on top and turns them into an API.

Model → migration → database

Django turns Python model changes into real tables

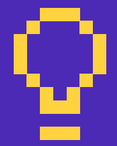


- Change a model, then run both commands again
- makemigrations writes a new numbered file and never edits old ones
- Migrations are committed to Git
- db.sqlite3 is rebuilt locally and is not committed

Field options and relationships

These rules appear in both the example and the exercise

Option	Meaning
`max_length`	required for CharField
`blank=True`	may be left empty in forms and validation
`null=True`	database may store NULL
`default=...`	value used when none is given
`unique=True`	no two rows share the value
`ForeignKey`	one parent can have many related rows



LIVE DEMO

Movies worked example

Watch the instructor build it end to end

models.py
fields, choices, __str__

migrations
makemigrations + migrate

admin.py
register and customize

ORM
create, read, update, delete

Afterwards, the Movies files are your read-only syntax reference

Reproducible setup

The environment should work the same way for every participant

```
# Windows
.venv\Scripts\Activate.ps1
# macOS
source .venv/bin/activate

python manage.py check
python manage.py runserver
```

- requirements.txt freezes the shared stack
- .gitignore keeps .venv, db.sqlite3 and .env out of Git
- check confirms Django can load the project
- A fresh terminal should start the server

Today is local. Production comes on Day 5.

HOUR 2 · EXERCISE

Library Catalog

Models, migrations, admin, and ORM practice

Core	Stretch	Time
Tasks 1, 2, 3A	Task 3B	~45 minutes

Result: 3 authors, 6 books, and a working admin

`Start in days/day-01/exercise/`

Exercise Task 1: complete Book

Use the existing Author model as your pattern

Field	Type	Rule
`title`	CharField	max_length=200
`isbn`	CharField	max_length=13, unique=True
`published_year`	PositiveIntegerField	required
`available_copies`	PositiveIntegerField	default=1
`author`	ForeignKey → Author	CASCADE, related_name=books

Also complete __str__ so a Book displays as its title.

Exercise Task 2: create the database

No code to write. Run the commands and read their output.

```
python manage.py makemigrations catalog
python manage.py migrate
python manage.py loaddata sample_data
```

- 0001_initial.py creates Author and Book
- migrate creates db.sqlite3 and tables
- loaddata installs 9 objects
- Optional: inspect catalog_book in DB Browser

Close DB Browser before running Django again, otherwise the database may be locked.

Exercise Task 3: admin and ORM

Finish the core admin task before the stretch queries

Core 3A · Admin
5 columns · author filter · title / ISBN search

Stretch 3B · ORM
count · filter · lookups · create · update · delete
Goal: ORM practice 5/5 passed

- Core acceptance: both models appear in admin
- Searching “clean” shows 3 books
- Filtering Martin Fowler shows 2 books

Short on time? Do the stretch after the project milestone.

BREAK

Short break

After the break: start your own Task Management API

HOUR 3 · PROJECT

Start the Task API

Apply the same ideas to your own repository · about 60 minutes

Create from the starter
new folder + new GitHub repo

Complete P1–P5
settings, model, __str__, admin

Build and verify
users, tasks, restart, push

Setup: create the project .venv and install requirements.txt.
Push: git init → check git status → add and commit → set origin → push main. Keep .venv/, db.sqlite3, and secrets out of Git.

Day 1 project increment

Five TODOs in three files

TODO	File	Purpose
P1	`config/settings.py`	register rest_framework and tasks
P2–P4	`tasks/models.py`	status choices, fields, __str__
P5	`tasks/admin.py`	columns, filters, search

Then run makemigrations and migrate, add two users and three tasks in admin, restart, and push.

The Task table

Build the data foundation for the API

Field	Type	Rule
`title`	CharField(200)	required
`description`	TextField	blank=True
`status`	CharField(20)	TODO / IN_PROGRESS / DONE
`due_date`	DateField	null=True, blank=True
`owner`	ForeignKey → User	CASCADE, related_name=tasks
`created_at / updated_at`	DateTimeField	auto_now_add / auto_now

In admin: create two users and at least three tasks with different owners and statuses.

Day 1 exit check

You are ready for Day 2 when all five pass

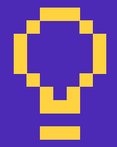
01	A fresh terminal activates .venv and starts the server
02	migrate reports “No migrations to apply”
03	Task fields and status choices match the table
04	Admin keeps records after a restart
05	GitHub has no .venv, db.sqlite3, or secrets

First push ran over? Finish it in the first 10 minutes of Day 2.

Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



Day 1 complete

You built the data foundation: project, app, model, migrations, admin, and saved records.

Next: serializers, viewsets, routers, CRUD endpoints, and validation